Visualização 3D Avançada de Árvores Arteriais CCO

Visibilidade, Iluminação e Técnicas Baseadas em Raios

BCC327 – Computação Gráfica | Trabalho Prático 2

Brenda Gabrielle Alves Nascimento
Matrícula 24.1.4011

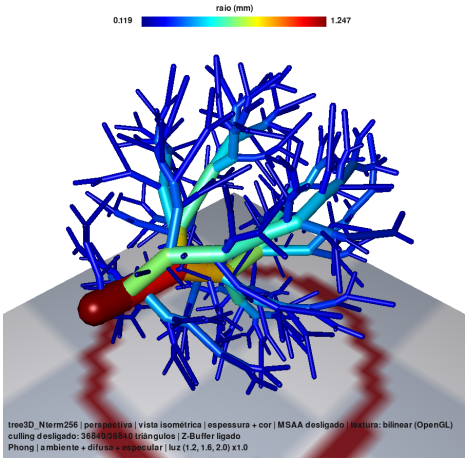
Departamento de Computação (DECOM)
Universidade Federal de Ouro Preto (UFOP)

2026.2 | Prof. Dr. Rafael Alves Bonfim de Queiroz

Roteiro da Apresentação

1. Contexto e Arquitetura
2. Visibilidade
3. Iluminação e Sombreamento
4. Transparência e Textura
5. Técnicas Baseadas em Raios
6. Recurso Adicional
7. Discussão e Conclusão

Do TP1 ao TP2



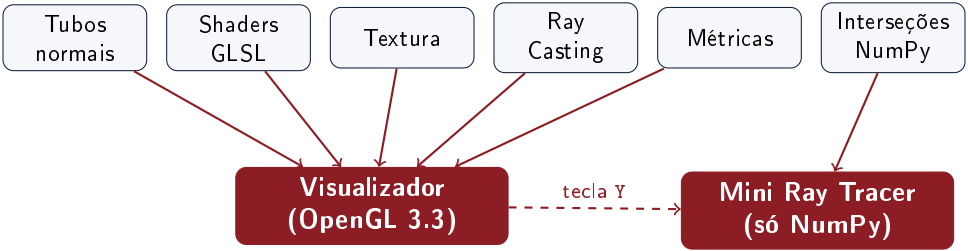
Problema

Uma árvore 3D com 511 vasos tem ramos sobrepostos em vários níveis de profundidade. Para ser interpretável, a imagem precisa de:

- **oclusão correta** entre os vasos
- **iluminação** que revele a forma cilíndrica
- **transparência** para ver o interior
- **interação**: consultar e isolar vasos

No TP1, iluminação e visibilidade eram caixa-preta do OpenGL.

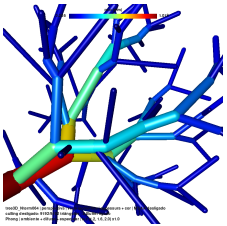
Arquitetura



Ideia central

O OpenGL rasteriza e interpola; as equações são nossas: matrizes, iluminação, culling, ordenação, filtragem e interseções.

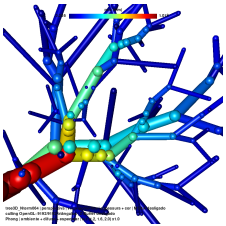
Back-Face Culling e Z-Buffer



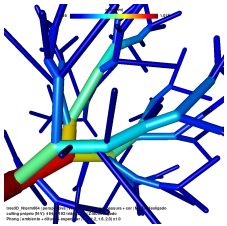
Z-Buffer



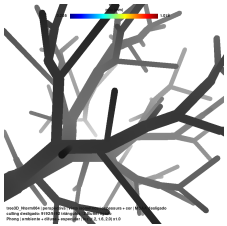
sem Z, sem culling



sem Z, culling GL



Z + culling próprio



profundidade

Culling próprio

descarta se $N \cdot V \leq 0$, com $V = -\text{centroide}$ (perspectiva)
4 546 de 9 192 triângulos (49,5%), imagem idêntica

Conclusão

faces de trás em vermelho: culling sozinho não resolve a oclusão *entre* vasos; só o Z-Buffer

Culling: próprio × OpenGL e o Z-Buffer

Dois critérios, mesmo resultado

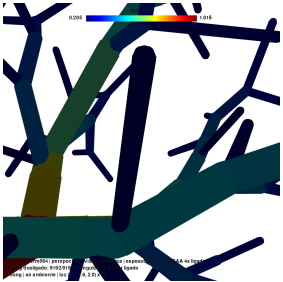
- **próprio**: geometria antes da projeção ($N \cdot V$)
- **OpenGL**: ordem dos vértices na tela (anti-horário = frente)
- exige faces orientadas: $(v_1 - v_0) \times (v_2 - v_0)$ para fora, garantido em `geometria.py`

Profundidade não linear

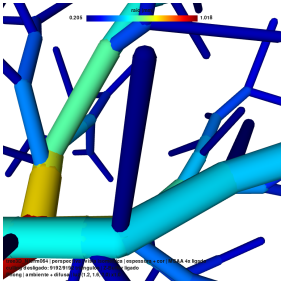
$$z = \frac{2nf}{(f+n) - (2d-1)(f-n)}$$

lido com `glReadPixels`, linearizado e normalizado ($n = 0{,}01$, $f = 100$): a precisão se concentra perto do plano near

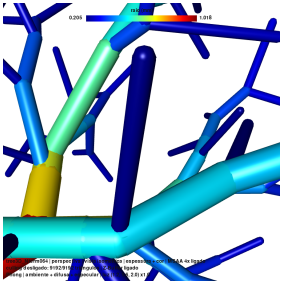
Blinn–Phong em shaders próprios



só ambiente



+ difusa



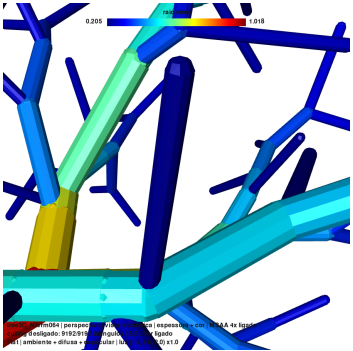
+ especular

Modelo de iluminação (espaço da câmera)

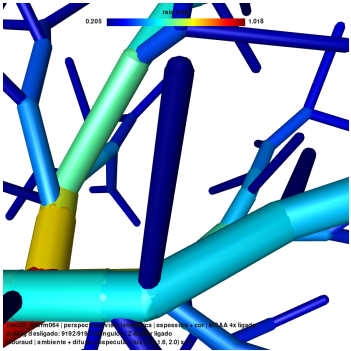
$$I = k_a\,\mathrm{c} + I_L\,[k_d\,\mathrm{c}\,\max(\mathrm{N}\cdot\mathrm{L},0) + k_s\max(\mathrm{N}\cdot\mathrm{H},0)^{\alpha}], \qquad \mathrm{H} = \frac{\mathrm{L}+\mathrm{V}}{\|\mathrm{L}+\mathrm{V}\|}$$

$k_a = 0{,}25,\ k_d = 0{,}75,\ k_s = 0{,}6,\ \alpha = 40$ | normais por $(M_{3\times3}^{-1})^T$

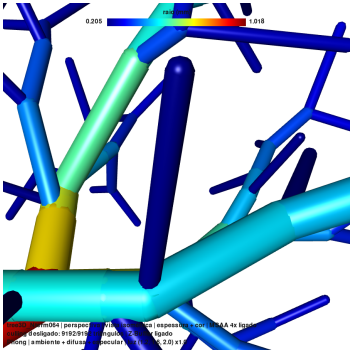
Flat × Gouraud × Phong



Flat: 1 vez por triângulo



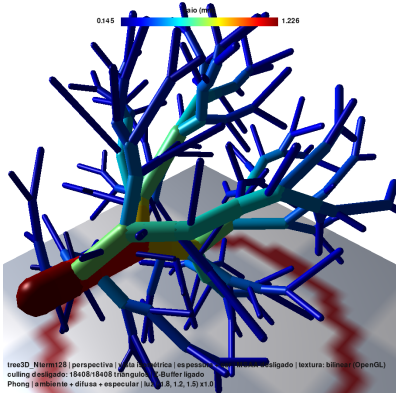
Gouraud: por vértice, interpola cor



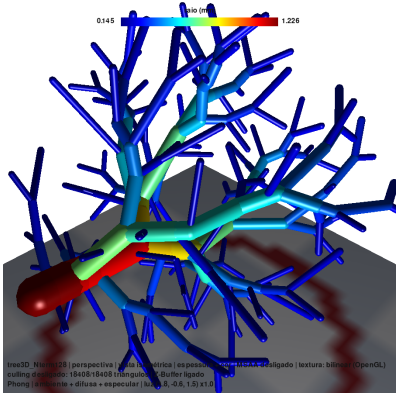
Phong: por pixel, interpola normal

- mesma equação, muda onde ela é avaliada (qualificador `flat`, vertex ou fragment shader)
  - Flat: tubos viram prismas de 12 lados
- Gouraud ≈ Phong nos vasos finos; nos grossos o Gouraud borra o brilho ao longo dos triângulos longos

Luz pontual móvel



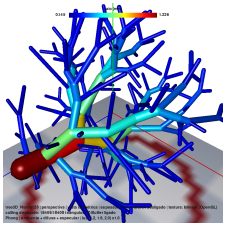
luz à esquerda



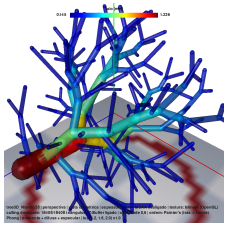
luz à direita

tecla `L`: setas e `PageUp`/`PageDown` movem a luz, `+`/`-` mudam a intensidade
a posição no mundo é levada ao espaço da câmera por V antes de ir ao shader

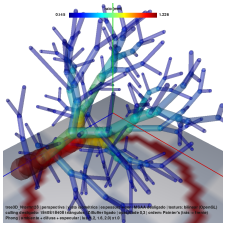
Transparência com Painter's Algorithm



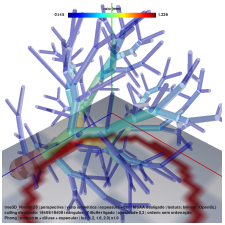
opaco



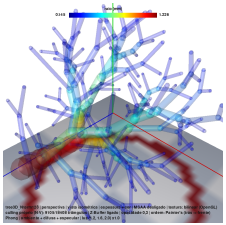
$\alpha = 0{,}6$



$\alpha = 0{,}3$, Painter



$\alpha = 0{,}3$, sem ordem

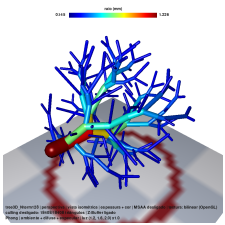


+ culling

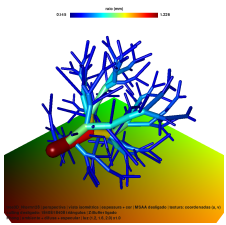
- $C = \alpha\, C_{\text{novo}} + (1 - \alpha)\, C_{\text{antigo}}$: depende da ordem
- triângulos ordenados pelo z do centroide, de trás para frente

- Z-Buffer testa, mas não grava (`glDepthMask`)
- com culling: 1 camada por tubo e 18 408 $\rightarrow$ 9105 triângulos

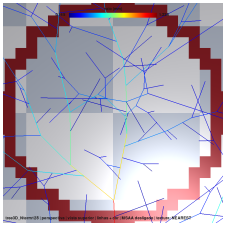
Textura e filtragem bilinear própria



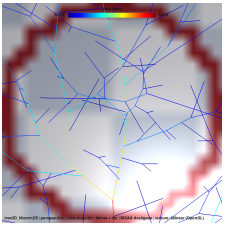
plano texturizado



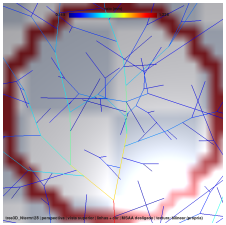
(u, v)



NEAREST



bilinear OpenGL



bilinear própria

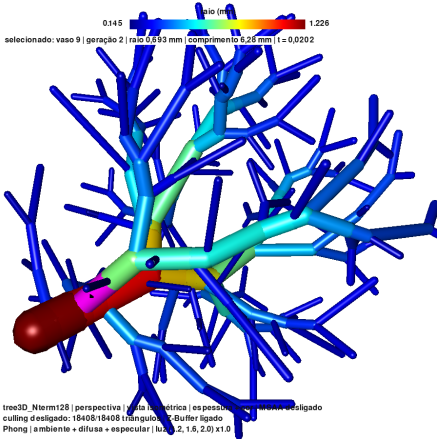
textura 32×32 gerada em NumPy
bilinear própria: 4 `texelFetch` (sem filtro) + interpolação

$$c = (1 - f_y)[(1 - f_x)c_{00} + f_x c_{10}] + f_y[(1 - f_x)c_{01} + f_x c_{11}]$$

Exemplo: $(u, v) = (0{,}245;\ 0{,}50)$

$f_x = 0{,}34$, $f_y = 0{,}50$
NEAREST: $(235, 235, 240)$
bilinear: $(212{,}5;\ 216{,}5;\ 227{,}0)$ = média exata

Seleção de vasos por Ray Casting



vaso 9 | geração 2 | raio 0,893 mm | $t = 0{,}0202$

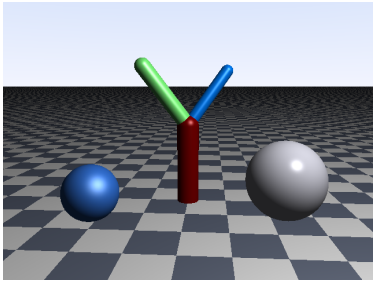
- pixel $\rightarrow$ NDC $\rightarrow (PV)^{-1}$: raio $\mathrm{O} + t\mathrm{D}$
- levado ao espaço do modelo por M^{-1}
- vaso = cilindro + 2 esferas; escolhe o menor $t > 0$

Raio–cilindro

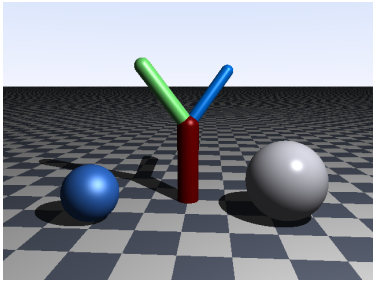
$k_2 t^2 + k_1 t + k_0 = 0$, com $\mathrm{m} = \mathrm{O} - \mathrm{A}$:
$k_2 = \mathrm{D}\cdot\mathrm{D} - (\mathrm{D}\cdot\mathrm{a})^2$, $k_1 = 2[\mathrm{m}\cdot\mathrm{D} - (\mathrm{m}\cdot\mathrm{a})(\mathrm{D}\cdot\mathrm{a})]$
$k_0 = \mathrm{m}\cdot\mathrm{m} - (\mathrm{m}\cdot\mathrm{a})^2 - r^2$, válido se $0 \le s \le h$

vetorizado: $\approx 0{,}6$ ms para os 1 023 vasos do Nterm512

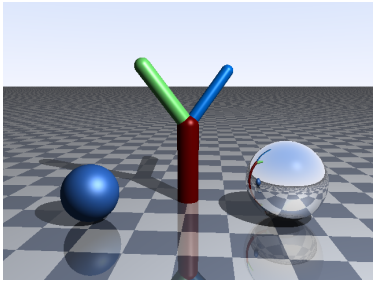
Mini Ray Tracer



1. Ray Casting



2. + raios de sombra

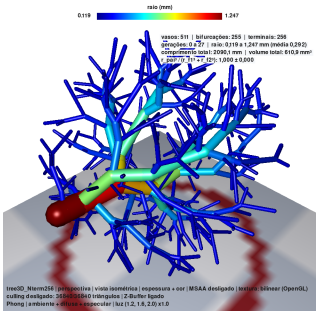


3. + raios refletidos

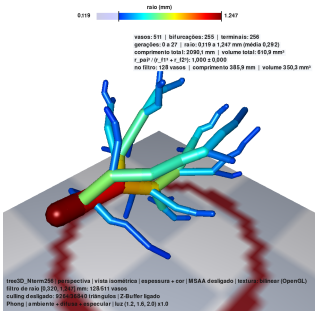
Modo (640×480)	**Primários**	**Sombra**	**Refletidos**	**Total**	**Tempo**
1. Ray Casting	307 200	0	0	307 200	0,76 s
2. + sombras	307 200	216 228	0	523 428	0,75 s
3. Ray Tracing	307 200	260 377	225 348	792 925	1,36 s

$R = D - 2(D \cdot N)N$, profundidade 3 | raios secundários partem de $P + \varepsilon N$ (evita *shadow acne*)

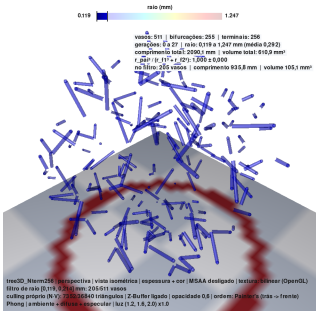
Métricas da árvore e filtro por raio



árvore completa



25% mais grossos



40% mais finos

- grafo + BFS a partir da raiz: geração de cada vaso
- comprimento, volume $\pi r^2 \ell$, bifurcações
- $r_p^3/(r_{f_1}^3 + r_{f_2}^3) = 1{,}000$ em todos os modelos: lei de Murray

Nterm256

25% mais grossos: 18,5% do comprimento, 57,3% do volume

40% mais finos: 44,8% do comprimento, 17,2% do volume

O que é nosso e o que é da biblioteca

Implementação própria	**Biblioteca**
Tubos, normais de vértice e de face, orientação	Janela, teclado, mouse, texto (pygame-ce)
Matrizes de modelo, câmera, projeção e normais	Compilação dos shaders, rasterização
Blinn–Phong, Flat, Gouraud, Phong (GLSL)	Interpolação dos atributos
Culling $N \cdot V$, linearização do Z-Buffer	Teste de profundidade, culling "OpenGL"
Painter's Algorithm	Composição alfa (`glBlendFunc`)
Textura, (u, v), bilinear própria	Amostragem NEAREST e bilinear (OpenGL)
Ray Casting, mini Ray Tracer, métricas	MSAA, VBOs, operações vetoriais (NumPy)

Discussão

Comparações

- culling = otimização; Z-Buffer = visibilidade
- Phong: brilho correto, custo absorvido pela GPU
- rasterização: interativa, sem sombras nem reflexões
- Ray Tracing: sombras e reflexões naturais, mas custo pixels $\times$ objetos $\times$ raios

Limitações

- Painter por centroide falha em interpenetrações (junções)
- ordenação refeita a cada quadro em Python
- Ray Tracer numa cena simplificada, sem antialiasing
- Nterm512 lento no WSL (renderização por software)

Conclusões

- pipeline programável com iluminação e sombreamento próprios
- visibilidade estudada com culling próprio, culling do OpenGL e Z-Buffer
- transparência ordenada e textura com filtragem bilinear própria
- raios na interação (seleção) e na síntese de imagens (Ray Tracer)
- métricas e filtro: 1/4 dos vasos concentra mais da metade do volume

Trabalhos futuros

Ray Tracer na árvore completa com BVH | transparência independente de ordem (OIT) | sombras por *shadow mapping*

Demonstração

Visualizador (`tree3D_Nterm128`)

- `F`, `D`, `B`, `V`: faces de trás, Z-Buffer, culling, profundidade
- `G`, `K`, `L`: sombreamento, componentes, luz
- `O`, `N`: opacidade, ordenação
- `T`, `U`: plano, filtragem

Raios e recurso adicional

- clique direito: seleção por Ray Casting
- `Y`: mini Ray Tracer, modos 1, 2, 3
- `I` + setas: filtro por raio
- `J`: painel de métricas

Referências

- SCHREINER, W.; BUXBAUM, P. F. Computer-optimization of vascular trees. *IEEE Trans. Biomedical Engineering*, 1993.
- FOLEY, J. D. et al. *Computer Graphics: Principles and Practice*. Addison-Wesley, 1990.
- CATMULL, E. *A Subdivision Algorithm for Computer Display of Curved Surfaces*. Tese, University of Utah, 1974.
- GOURAUD, H. Continuous shading of curved surfaces. *IEEE Trans. Computers*, 1971.
- PHONG, B. T. Illumination for computer generated pictures. *CACM*, 1975.
- BLINN, J. F. Models of light reflection for computer synthesized pictures. *SIGGRAPH*, 1977.
- NEWELL, M. E.; NEWELL, R. G.; SANCHA, T. L. A solution to the hidden surface problem. *ACM Annual Conference*, 1972.
- APPEL, A. Some techniques for shading machine renderings of solids. *AFIPS*, 1968.
- WHITTED, T. An improved illumination model for shaded display. *CACM*, 1980.
- MURRAY, C. D. The physiological principle of minimum work. *PNAS*, 1926.

Comandos de Execução

```
pip install -r requirements.txt
python src/visualizador.py data/tree3D_Nterm128.vtk
python src/raytracer.py            # ou tecla Y no visualizador
python src/metricas.py data/tree3D_*.vtk
python src/gerar_figuras.py        # regera as figuras
```

Observação

O README traz instalação, comandos de teclado e mouse e o roteiro de teste.

Obrigada!

Perguntas?

Brenda Gabrielle Alves Nascimento | 24.1.4011